| USN: | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

**School of Computer Science and Engineering**
**Minor in Robotics and Industrial Automation**
**CIE - 3: Project**
**Academic Year** 2025 – 2026

| **Course:** Introduction to Robotics | **Course Code:** CS2110 | **Semester:** III |
|---|---|---|
| **Date:** 5 Nov. 2026 | **Duration:** Presentation | **Max Marks:** 15 Marks |

# Project Title:

## Implementation and Simulation of Forward and Inverse Kinematics for a Differential Drive Robot and a Planar Manipulator Arm

| S.No | Problem Statement | Marks | Level | CO |
|---|---|---|---|---|
| 1 | **a) Objective:**<br>To *implement, simulate and critically analyse* the forward and inverse kinematics of two robots using Python and Pygame: (i) a **differential drive mobile robot** and (ii) a **planar two-link manipulator arm**. Through this project you will:<br>(i) get first-hand exposure to how kinematic models are turned into working code;<br>(ii) visualise the simulation to see how wheel / joint commands become motion;<br>(iii) use your implementation to perform basic tasks: for the *differential drive*, use forward kinematics (FK) to estimate pose and inverse kinematics (IK) to drive from point A to point B; for the *manipulator*, use FK to move the end-effector from point A to point B.<br>Marks are awarded mainly for your **depth of understanding** and your ability to **explain** what you have built during the presentation and viva.<br>**b) Starter code provided:**<br>You are given two *partially working* Python–Pygame programs: a differential drive simulator (`diffdrive_starter.py`) and a planar 2R arm simulator (`arm_starter.py`). Both open a window, draw the robot and run a keyboard-controlled loop.<br>• **Already working:** window and main loop, drawing helpers, keyboard control, the robot state, the basic FK update of the differential drive pose, and the FK function of the arm.<br>• **Left for you:** trajectory logging and drawing, the autonomous behaviours, on-screen display of state, parameter handling, and the analysis tasks below. Places to extend are marked `# TODO`.<br>Simulation units: metres, radians, seconds, drawn with a fixed scale (e.g. 100 px = 1 m). Be careful with the pixel $y$-axis, which points downward.<br>**c) Kinematic models (notation used in this statement):**<br>*Differential drive* with wheel radius $r$, wheel base $L$, wheel angular speeds $\omega_L, \omega_R$ and pose $(x, y, \theta)$:<br>$v = \frac{r}{2}(\omega_R+\omega_L), \quad \omega = \frac{r}{L}(\omega_R-\omega_L), \quad \dot{x} = v\cos\theta,\ \dot{y} = v\sin\theta,\ \dot{\theta} = \omega$<br>$\text{IK:} \quad \omega_R = \frac{2v+\omega L}{2r}, \qquad \omega_L = \frac{2v-\omega L}{2r}, \qquad \text{circle: } R = \frac{v}{\omega}$<br>Go-to-goal controller for target $(x_g, y_g)$: with $\rho = \sqrt{(x_g-x)^2+(y_g-y)^2}$ and $\alpha = \text{wrap}\big(\text{atan2}(y_g-y, x_g-x)-\theta\big)$, use $v = k_\rho\rho$ and $\omega = k_\alpha\alpha$ (with saturation), then convert to wheel speeds using the IK above.<br>*Planar 2R arm* with link lengths $l_1, l_2$ and joint angles $\theta_1, \theta_2$:<br>$x = l_1\cos\theta_1 + l_2\cos(\theta_1+\theta_2), \qquad y = l_1\sin\theta_1 + l_2\sin(\theta_1+\theta_2)$<br>$\text{IK:} \quad \cos\theta_2 = \frac{x^2+y^2-l_1^2-l_2^2}{2l_1l_2}, \qquad \det J = l_1l_2\sin\theta_2$ | 15 | L4, L5 | CO4 |

| S.No | Problem Statement | Marks | Level | CO |
|---|---|---|---|---|
| | **d) Tasks to be completed**<br>**Part 0 – Run and understand.** Run both programs. In your report, point to the exact lines where each kinematic equation above appears, trace *one* simulation time-step by hand and match it with the on-screen values, and explain the role of $\Delta t$ and of unit conversion (m $\leftrightarrow$ px).<br><br>**Part 1 – Differential drive robot.**<br>1.1 **FK and dead reckoning.** Display $(x, y, \theta)$, $v$, $\omega$ on screen and draw the trajectory. For four constant command pairs — (i) $\omega_L = \omega_R$, (ii) $\omega_L = -\omega_R$, (iii) one wheel stopped, (iv) wheel speeds that trace a *circle of radius* $R$ — predict the pose after $T$ s analytically, run the simulation, tabulate predicted vs. simulated pose and explain every discrepancy (consider the integration scheme and $\Delta t = 0.1$ vs. 0.01 s). For (iv), check how well the path closes after one full revolution.<br>1.2 **IK: move from A to B.** Place the robot at pose A and drive it to goal B (chosen by mouse click or typed coordinates), stopping within a tolerance $\varepsilon$. Compute wheel velocities through the IK equations. Show start/goal markers, trajectory and live wheel speeds, and plot $v$, $\omega$ and distance-to-goal against time.<br>1.3 **Parameter study.** Vary at least $k_\rho$, $k_\alpha$, wheel base $L$, maximum wheel speed (saturation) and the initial heading error (including a goal directly *behind* the robot). Overlay the trajectories and tabulate path length, time-to-goal, peak $\lvert\omega\rvert$ and final error. Explain each observation using the equations.<br><br>**Part 2 – Planar manipulator arm.**<br>2.1 **FK verification.** For three joint configurations, compute the end-effector position by hand and compare it with the value your simulation displays.<br>2.2 **Move end-effector from A to B using FK.** Points A and B are given as joint configurations $\mathbf{q}_A, \mathbf{q}_B$. Implement time-parametrised motion $\mathbf{q}(t) = \mathbf{q}_A + s(t)\,(\mathbf{q}_B - \mathbf{q}_A)$, $s : 0 \to 1$ over $T$ s, and at every frame compute the end-effector with FK. Draw the end-effector trail and markers for A and B, and report the path length and the maximum deviation from the straight chord A–B.<br>2.3 **Parameter study.** Change $l_1, l_2$, the duration $T$ and the time scaling $s(t)$ (linear, cubic, trapezoidal). Compare trail shape and end-effector speed profile, and explain why equal-time joint interpolation gives a curved Cartesian path.<br>2.4 **Workspace and singularities.** Sample the joint space to plot the reachable workspace, identify its boundaries, and relate the configurations $\theta_2 = 0, \pi$ to $\det J$.<br>2.5 **IK extension.** Implement analytic IK for a target $(x, y)$: both elbow solutions, and detection of unreachable points. Move A$\to$B along a *straight line in Cartesian space* using IK at every step, and compare it with 2.2.<br><br>**Part 3 – Live code modification (during demo).** The examiner will ask for one or two changes to your own code, to be made and explained on the spot. Examples (not limited to these): stop at B with a specified final heading; visit waypoints A$\to$B$\to$C; add wheel-speed saturation or Gaussian noise on wheel speeds and show the drift; trace a figure-eight; add joint limits to the arm; switch elbow-up to elbow-down; change the time-scaling; add a third link. | | | |

| S.No | Problem Statement | Marks | Level | CO |
|---|---|---|---|---|
| | **Part 4 – Analysis and evaluation.** Answer in the report; be ready to defend in the viva.<br>4.1 Why does dead reckoning drift on a real robot? Which sources of error are absent in your simulation, and how does $\Delta t$ affect the result?<br>4.2 Compare the Euler pose update with the exact arc update. For which values of $\omega\,\Delta t$ does the difference matter?<br>4.3 Why can a differential drive robot not move sideways? Relate this to the constraint $\dot{x}\sin\theta - \dot{y}\cos\theta = 0$ and to its degrees of freedom.<br>4.4 Where is the instantaneous centre of rotation for different ratios $\omega_R/\omega_L$? When does the robot turn in place?<br>4.5 Evaluate your go-to-goal controller: what happens when $\alpha \approx \pm\pi$? Are your chosen gains a good compromise between speed and smoothness? Propose and justify one improvement.<br>4.6 Joint-space vs. Cartesian straight-line motion of the arm: compare their advantages and drawbacks, and judge which suits pick-and-place and which suits welding.<br>4.7 What happens to joint speeds when a Cartesian straight line passes near a singularity?<br>4.8 Compare the two robots: degrees of freedom, constraints, closed-form vs. controller-based IK, uniqueness of solutions.<br><br>**e) Simulation requirements.**<br>• Robot drawn as a simple shape (rectangle/circle with a heading marker; links and joints for the arm).<br>• Current trajectory (drawn path), start and goal markers, and an on-screen display of the state and commands.<br>• **Bonus (optional):** a mode selector (keyboard control, go-to-goal, circular motion, arm A→B).<br>**f) Deliverables.**<br>(a) Python script(s) (.py) implementing the simulations.<br>(b) Properly commented source code explaining (i) the kinematic equations used and (ii) the motion control logic.<br>(c) A project report (format attached) with derivations, screenshots of the code and of the simulation output for both robots, the tables and plots from Parts 1–2, and the answers to Part 4.<br>(d) Presentation and demo, followed by a viva.<br>**Note:** You may consult references, but you must be able to explain and modify every line of your code live. Code you cannot explain will be treated as not your own, and marks will be reduced. | | | |

## Marking Scheme:

1 Project Report 5 Marks
2 Viva 5 Marks
3 Demo 5 Marks

**Evaluation focus (depth of understanding):**

| Component | What is assessed | Marks |
|---|---|---|
| Project Report | Correct derivations and mapping of equations to code; complete results, tables and screenshots; quality of analysis in Parts 1, 2 and 4 | 5 |
| Viva | Explaining equations and code lines; predicting behaviour for "what if" parameter changes; critiquing and justifying design choices (L4–L5) | 5 |
| Demo | Working simulations for both robots; Part 3 live modification; changing parameters to obtain a requested trajectory characteristic | 5 |

## Course Outcomes:

CO1: Apply configuration space and workspace concepts to analyze robotic motion capabilities using Grubler's formula.
CO2: Implement rigid body transformations using homogeneous coordinates and rotation matrices.
CO3: Develop kinematic models for robotic manipulators using Denavit-Hartenberg (DH) and POE representations.
CO4: Construct motion equations for mobile robots considering kinematic constraints and degrees of freedom.

## Marks Distribution

| L1 | L2 | L3 | L4 | L5 | L6 | CO1 | CO2 | CO3 | CO4 |
|---|---|---|---|---|---|---|---|---|---|
| – | – | – | 8 | 7 | – | – | – | – | 15 |

*****